Sentinel Logic

Collusion Detection and Response: Oversight That Outlives the Transcript[1]

Mikey Whiston
Security Researcher & Systems Architect
Sentinel Logic Ltd
mikey@mikeywhiston.dev

[1]Research conducted at the Swarm Alignment Hackathon, November 2026

Abstract

We recorded 42 file interactions between model-driven agents sharing one sandbox. A monitor reading only their declared transcript detected none of them, including the episodes in which every tool call was declared accurately and the binary answering to the declared name was hostile. Oversight of agent swarms reads a record the agent itself writes, whether that record is a transcript, a tool call, or a thinking trace. The failure is structural, so we build CDR, the oversight that does not read it. A kernel probe attributes every file touch to an agent by cgroup inode, an identifier the agent cannot choose, and emits an edge when two agents touch the same file. A policy then decides whether that edge was sanctioned, so the detector records what agents did without needing to infer why. Across 80 scripted test cases spanning 19 filesystem channels, we recover 88 of 88 interactions with zero false edges, and no episode in a benign control group of 40 produces an unsanctioned edge. Agents colocated in one container are separable when the harness partitions them into cgroups. The mechanism we use covers storage channels that the kernel reports as syscalls; timing channels, network egress and asynchronous I/O fall outside coverage.

1 Introduction

A modern agent with access to a shell is the most useful type of agent, but it is also the least observable one. It can write files, replace binaries, spawn processes, and manufacture its own evidence, so every record of what it did is self-reported rather than observed. That is not a hypothetical: we ran eighteen episodes of model-driven agents in a shared sandbox and the kernel observed 42 file interactions between them, of which a monitor reading only the declared tool calls detected none.

Existing oversight for agents reads exactly those records. Observability platforms log tool calls inside the runtime loop, harnesses score the transcript, and supervisors read what the agent says it is doing. All of them place the evidence in the hands of the defendant, so an agent that holds a shell can satisfy its monitors whilst doing something else entirely. The failure is sharper than ordinary lying, however. In our *laundering* episodes the agent declares `pytest -q` and runs it, and the binary answering to that name has been swapped for one that performs the covert write. Declaration and execution both happened, reconciliation reports a match, and the exchange never appears in any record the agent produced. Each of these systems produces telemetry about one subject. None of them reports what two agents did to each other.

We move the evidence out of the agent's reach. We call the resulting system CDR. We assume the agent has root inside its container but cannot change its cgroup membership, attach a probe, or modify the kernel. A kernel probe on the host records each file touch with the cgroup inode of the process that made it, resolves the object by device and inode inside the container's mount namespace, and emits an edge when two different agents touch the same object. An agent can rename itself, rewrite its transcript, or replace its own tooling, but it cannot alter its position in the cgroup hierarchy, the inode it touched, or the kernel's record of its syscalls. A policy file then decides whether an edge was sanctioned; the detector records all interactions and never infers the agent's intent.

On 80 scripted episodes over 19 filesystem channels, we recover 88 of 88 interactions with zero false edges, and 40 benign episodes produce no unsanctioned edges. On the live suite, the kernel records 42 interactions whereas a transcript monitor (representing a supervisor model) recovers none.

Our main contributions for this paper are:

1. **An agent identity and object model that cannot be altered by the subject.** Agent identity is a cgroup inode and the object key is the kernel's (device, inode) pair, so coverage follows from the mechanism rather than a finite list of applicable channels. Syscalls are categorised into mutators and sensors, so a channel we never anticipated still works if the kernel reports it as a syscall naming an object.

2. **A measure of how much oversight transcript-only monitoring provides.** Across a scripted suite with pre-registered ground truth and a live suite of model-driven agents, transcript-based monitoring recovers none of the 42 observed interactions, while the kernel recovers all 88 of 88 in the scripted suite with no false edges.

3. **A verdict model that separates observability from interpretation.** Collusion is strictly defined as an unsanctioned edge, so the detector reports what agents did and the policy decides whether it was permitted, which keeps the mechanism auditable and its alarms revisable.

2 Related Work

3 Methods

Figure 1 summarises the arrangement the rest of this section builds. Everything inside the dashed container is controlled by the agent and can be modified by it, whereas everything outside is recorded by the kernel. The mechanism exists so that a verdict depends only on the right-hand side, and every design choice below is an attempt to move a piece of substantiated evidence across that boundary.

Identity. No field of an agent's identity is left self-reported. A cgroup is the kernel's own grouping of processes, fixed when the process is created and exposed as a directory tree. An agent is the cgroup v2 inode it belongs to, read via `bpf_get_current_cgroup_id()` at syscall entry and resolved to the nearest registered ancestor in the sandbox subtree. A container-per-agent deployment needs no further configuration for CDR to function. Where several agents share a single container, however, the harness must create child cgroups and declare them to the loader out of band so that the agents become separable. We first accepted any nested cgroup whose name matched a designated prefix, and we rejected it after reviewing the security implications: an agent with the capability to create cgroups could adopt a neighbour's name, or create several identities and split its own behaviour across them, which manufactures edges between two halves of one attacker. A name the subject chooses cannot be an identity, so declarations are supplied by the deployment and nested cgroups that were not declared resolve to their nearest declared ancestor. An agent therefore cannot acquire an identity by creating a directory.

Objects. Objects are joined on what the kernel knows, not on what the agent called them. Events are keyed on the pair $(device, inode)$ rather than on the inode alone, because inode numbers repeat across containers that share image layers. In our testbed, the shared volume collapses to a single key, $29/2693466$, for every agent, while the same binary in three containers appears as inode 2604173 under devices 85, 74 and 82. Keying on the inode alone would report every agent that launches `bash` as sharing an object with every other, whereas keying on the pair separates them without any knowledge of file names.

Capture. The probe does not need to know which channels exist in order to spot them. Ring-0 holds no per-syscall knowledge. For every syscall it captures, the probe records the syscall number, its arguments, the cgroup, and up to two user strings, chosen by a bitmap the loader derives at startup from the kernel's own tracepoint metadata. An argument is marked when it is typed `char *` and its name indicates a path. Vectors such as `execve` arguments are read element-wise onto a separate payload ring so they are not truncated. Adding a syscall is a map insertion instead of a recompilation. Syscalls outside the bitmap are counted per-agent rather than discarded, so an unmodelled channel surfaces as a frequency anomaly against the benign baseline instead of as silence. Local sockets illustrate the generality of this approach: the name of an `AF_UNIX` endpoint sits behind a pointer to a structure rather than in a path argument, so the probe reads the 2-byte address family and follows `sun_path` only when the family is `AF_UNIX`. No channel is enumerated; syscalls are.

Resolution. A path means nothing until it is resolved inside the container that wrote it. The loader resolves each one with `openat2` using `RESOLVE_IN_ROOT` and `RESOLVE_NO_MAGICLINKS`

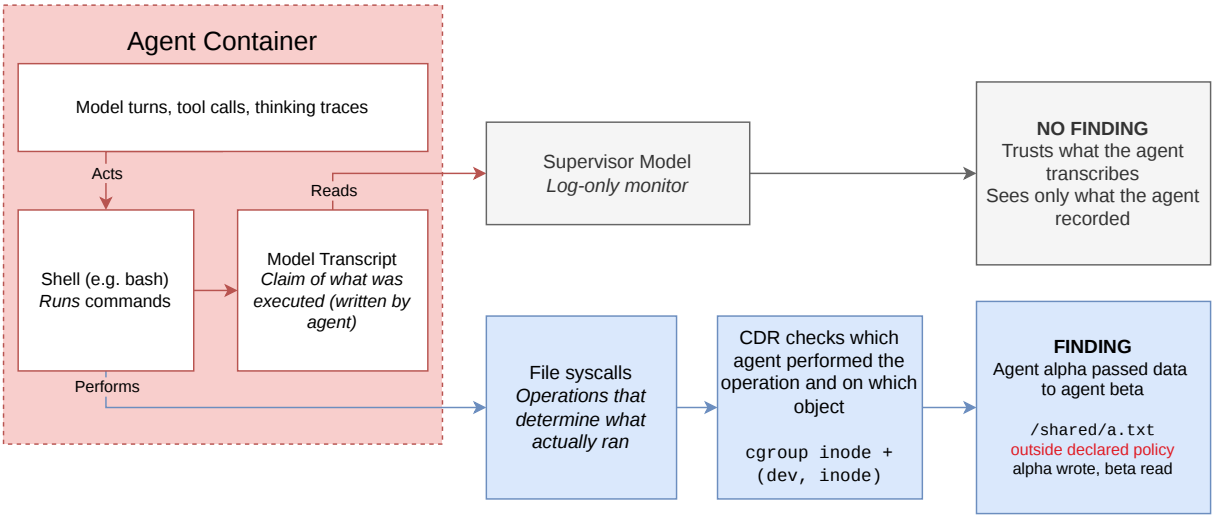


Figure 1: Inside the dashed container, everything is written by the agent and can be rewritten by it, so the transcript is a claim. The supervisor model has no input but that claim, and cannot separate a forged transcript from an honest one that simply omits the action. The kernel witnesses the syscall instead, resolves which agent performed the operation and on which object, and emits the edge when two agents touch the same object. The object is joined on device and inode, and the path is shown only for the reader. Agents `alpha` and `beta` are two containers in a sandbox slice.

against an `O_PATH` descriptor for the agent's `/proc/<pid>/root`, then `fstats` the result. Both flags exist because of observed failures in testing: an absolute symlink target was otherwise resolved against the host's root, and `..` at the container root escaped to a host inode shared by every container, each producing false edges. Results are cached by $(agent, path)$ and invalidated on `O_CREAT`, since a delete-and-recreate can be served a different inode. Because `bind` and `creat` create their object during the call, resolution at entry necessarily fails (the file does not exist yet), so such an event is retried a finite number of times, and a cached root that outlives its process falls back from the open descriptor to the cached prefix and finally to the calling thread's own `/proc/<tid>/root`.

Attribution. Every syscall can be traced to the declaration that authorised it, as a containment claim rather than a proof. The `sched_process_fork` tracepoint yields parent and child, and `execve` yields the argument vector. Together, they form an exec forest for a given agent, where any declared tool call can be bound to the node whose effective command matches it, after unwrapping one shell level and comparing basenames, and that node's subtree is the delegated action. Every object touch carries a thread-group identifier, so that an edge can be walked back to the declaration that authorised it.

Verdicts. The detector records what agents did; the policy decides whether it was permitted. Declarations are reconciled against executions into four classes: `match`, `mismatch` for a declaration with no accompanying execution, `omission` for silence, and `surplus` for an execution with no covering declaration. Separately, each edge is evaluated against a sanction set over paths, yielding sanctioned or unsanctioned. Collusion is defined as an unsanctioned edge. Syscalls are classified as mutators or sensors, and a syscall we have not classified defaults to sense, so any novel communication medium participates as an undirected adjacency rather than being dropped.

4 Results

5 Discussion and Limitations

Limitations

Future Work

6 Conclusion

Code and Data

LLM Usage Statement